One Stream, Many Windows: Soft Targets for Consolidating Fast and Slow Conversions

Anonymous

Anonymous

Abstract. Conversion pipelines in advertising are split by the delay time a conversion is observable: an upper-funnel onsite conversion is observable within minutes of the click, while a down-funnel offsite conversion takes a day or more. Each objective defines an *attribution window*, the elapsed time within which a conversion counts. Conventionally a training pipeline's join delay is the same as conversion attribution window. Tying the two together forces every objective onto its own pipeline, each as stale as the attribution window it waited for. We ask whether they can be set independently. Naively writing training records sooner makes every conversion still in flight become a negative example, and a streaming pipeline that visits each click at most once cannot revise them afterwards. Yet most published corrections do exactly that: of 9 methods we reproduce, 7 write more than one training record per click, up to 4.00. On the Criteo Delayed Feedback Dataset we compare three targets across 5,520 runs, four served capacities, three join delays, two attribution windows and two teacher architectures. Writing the record early and replacing only the negatives with a teacher model's estimate beats waiting for the matured label in 23 of 24 configurations, by 1.90% at seven days and 0.70% at one, which is 76% and 79% of what a label-peeking oracle attains. It is the strongest method obeying the single-visit constraint.

Keywords: Conversion rate prediction · Delayed feedback · Online advertising · Knowledge distillation · Streaming training pipelines.

1 Introduction

Advertising systems predict several types of conversion at once, and those types differ sharply in how long they take to observe. A website view or an add-to-cart is recorded within minutes of the click; a purchase, an app install or a submitted lead may take a day or a week. Each objective is defined with an *attribution window*, the elapsed time within which a conversion is credited to the click. Conventionally that window also determines the *join delay*, the age at which the click's training record is written to the stream: the pipeline waits until the label is knowable, then emits the record. This conversion has two disadvantages: A model for a slow objective trains only on data at least an attribution window old, so it never sees fresh data during online training. And a system that ranks

several objectives jointly must either maintain one pipeline per objective or delay all of them until the slowest window closes, imposing the staleness of the slowest funnel on the fastest.

Neither those two disadvantages are intrinsic to conversion rate modeling. While the attribution window is fixed by the business definition of a conversion, the join delay is a pipeline choice. We ask what happens when the two are set independently, and specifically whether an objective with attribution window of days can be trained from records joined within minutes.

The obstacle is that an early-joined record is wrong: every conversion still in flight when the record is written is recorded as a negative. A substantial literature corrects this censoring bias [1,11,10,2,4], but most corrections revise the record afterwards, by duplicating the click when its outcome lands, by relabelling it, or by adding prediction heads at future horizons. A streaming pipeline that appends each record once and trains on it once can do none of these. We state that restriction as a *single-visit* constraint.

Our idea is this censoring bias can be mitigated by the prediction from a model that has seen trained on the matured outcomes in the conventional way, i.e. *a teacher model*. Such a teacher model is often readily available in production, where large ranking models are already trained across objectives on matured data [8]. The contributions of this work are: (1) a separation of the join delay from the attribution window, with the causality and single-visit constraint a deployable pipeline must satisfy. After auditing 9 published methods, 7 methods violate single-visit by writing more than 1 record per click; (2) evidence that writing the record early and replacing only its observed negatives with teacher model predictions beats waiting for the matured label; (3) the finding that at small served capacities the hard label can be dropped altogether, since a training pipeline consisting only of the teacher's estimations as labels can beat the conventional training pipeline with delayed join same as attribution windows, which can benefit early stage, such as retrieval, where model complexity is low.

2 Problem Setting and Proposed Approach

A click i occurs at time C_i with features x_i, then the pipeline waits for conversions for a window D_i. The attribution window v fixes the target $Y_i^{(v)} = \mathbf{1}[D_i \le v]$, knowable only at $C_i + v$. The join delay o fixes when the record is written, so the label available at that moment is

$$Y_i^{(o)} \;=\; \mathbf{1}[D_i \le o] \;\le\; Y_i^{(v)}. \tag{1}$$

Setting $o = v$ reproduces the conventional pipeline. Setting $o \ll v$ is what we want in order to consolidate fast and slow conversions into the same training pipeline.

What a deployable pipeline must satisfy. We cannot train on the conversions landed in the future at any moment, which is a straightforward **causality rule**. In addition, we have the **single-visit** constraint: Let n_i be the number of times

a click i enters the served model's loss over its lifetime. This practical constraint used widely in the production systems requires $\max_i n_i \leq 1$.

Which published methods comply. We reproduced 9 published methods from scratch under a single protocol and counted the records each one's served model consumes per distinct click. 7 write more than one. At $v = 1\,\mathrm{d}$ the counts are FTP [5] 4.00, MISS [6] 3.05, PI [13] 2.00, DDFM [3] 1.89, DFM [1] 1.33, DEFUSE [2] 1.03, ES-DFM [10] 1.03 . Only FSIW, ULC satisfy the single-visit constraint. Of the two, FSIW corrects the loss and leaves the target itself untouched, while ULC changes the training target.

Proposed methods. We studied four types of pipeline target generation methods: FRESH writes at $\Delta = o$ with $\tilde{y}_i = Y_i^{(o)}$, filling the deficit with nothing. WAIT writes at $\Delta = v$ with $\tilde{y}_i = Y_i^{(v)}$, which is the conventional method and our comparison baseline. CORRECT-DISTILL writes at $\Delta = o$, keeps the observed labels and softens only the observed negatives with a teacher model's prediction.

$$\tilde{y}_i \;=\; Y_i^{(o)} + (1 - Y_i^{(o)})\, w^T(x_i), \qquad w^T(x) \approx P\Big(Y^{(v)}{=}1 \;\Big|\; Y^{(o)}{=}0,\, x\Big)\,. \tag{2}$$

An observed positive stays exactly 1, and only the uncertain mass moves. This is the functional form used by ULC [9]; the two differ in how w^T is estimated. DISTILL-ONLY goes further and replaces both all observed labels with a teacher model's prediction. A teacher model in both DISTILL-ONLY and CORRECT-DISTILL is trained with the conventional WAIT method. Finally, ORACLE writes at $\Delta = o$ but labels the record $Y_i^{(v)}$. It violates the causality rule and is not deployable. We include it because it bounds what any causal correction could recover.

3 Experiments and Results

3.1 Experimental Setup

Data and protocol. We use the public Criteo conversion logs released with [1]. Clicks span 60 days. Days $[0, 30)$ initialise the model and days $[30, 60)$ form the hourly online training period, divided into 719 hourly rounds yielding 8,196,131 scored clicks at a base conversion rate of 0.1361. Learning rates are selected per arm and per cell from a seven-point grid on a held-out slice of the initialisation period, then pinned across seeds. The teacher model's rate is fixed and decoupled from the serving model's.

Model architecture. In the model arch backbone, the eight integer fields enter a dense arm as $\log(1+x)$ paired with an observed-mask bit, normalised and passed through a two-layer perceptron to a projection vector. The nine categorical fields are hashed per field and embedded, giving a token per field. Tokens and the dense projection are then consumed by a stack of Wukong interaction layers [12], then output the final prediction.

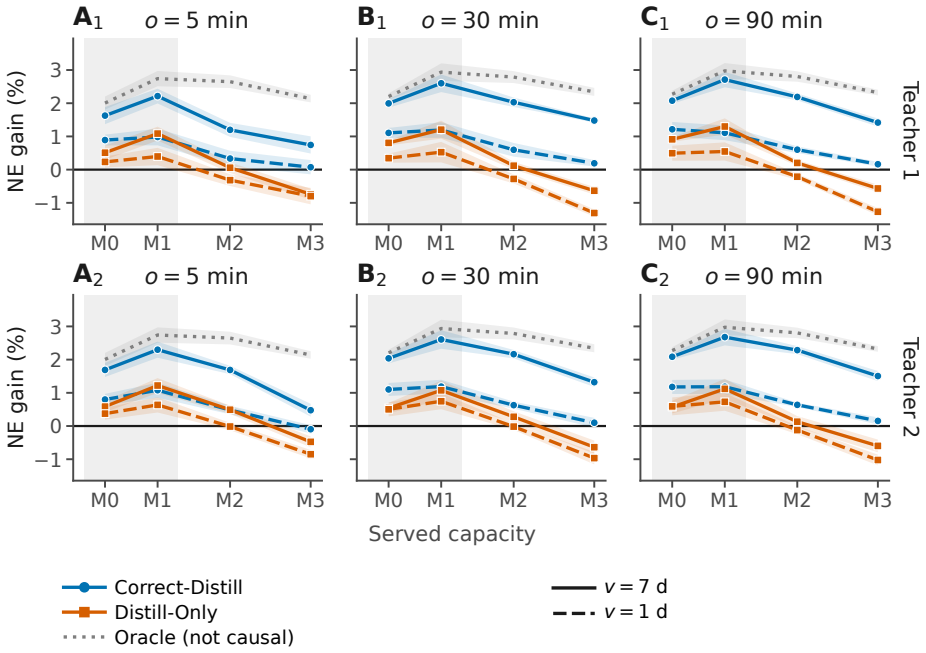


Fig. 1. Relative NE reduction against WAIT as a function of served capacity, for different join delay o, and two different teacher models. Shading on a curve is the standard deviation of the per-seed paired differences.

Different setups and evaluation metric. Two attribution windows ($v \in \{1, 7\}$ days), three join delays ($o \in \{5, 30, 90\}$ minutes), four different serving model complexities (M_0 to M_3), and two different teacher models are used. Across the four complexities the hash width per categorical field grows $36 \to 256 \to 1{,}280 \to 16{,}384$ and the embedding dimension widens $4 \to 4 \to 8 \to 8$, roughly an order of magnitude of parameters per step, 1,890 to 1,184,439 parameters and 0.001 to 0.0095 MFLOPs per inference. For the two teacher models, *Teacher 1* places 8-dimensional embedding tables under a two-layer perceptron, giving 9,456,897 parameters at 0.039 MFLOPs. *Teacher 2* places 48-dimensional tables under a deep interaction trunk, giving 57,272,515 parameters at 1.593 MFLOPs. Running both is what lets us report the method's sensitivity to the choice of teacher rather than to one arbitrary configuration of it. For each step, we ran five to ten seeds for statistical analysis. We report normalized entropy and calibration against mature $Y^{(v)}$ for all methods.

3.2 Findings

Decoupling the join delay from the attribution window improves accuracy. Writing the record early and replacing only its observed negatives with the teacher's estimate beats WAIT in 23 of 24 cells, by 1.90% on average at $v = 7\,\mathrm{d}$, where

Table 1. All reproduced methods at $v = 1\,\mathrm{d}$ and $o = 90$ minutes, the operating point closest to a large-scale production setting, at the smallest and largest served capacities. NE and calibration error are absolute and lower is better; ΔNE is the per-seed paired reduction against WAIT in percent, higher is better, reported with the half-width of its 95% t interval. The replicate is a seed, five per arm and ten for WAIT, and a (teacher, seed) pair for the two arms in bold, which are ours and ran under both teachers. ‡ORACLE writes one record but labels it with the matured outcome, so it violates causality and is an upper reference rather than a method.

	M_0 (smallest)			M_3 (largest)		
Method	NE	Cal.	ΔNE (%)	NE	Cal.	ΔNE (%)
Single visited: yes						
WAIT	0.7865	0.0578	ref.	0.7197	0.0743	ref.
ORACLE‡	0.7769	0.0489	$+1.23 \pm 0.28$	0.7169	0.0677	$+0.40 \pm 0.09$
CORRECT-DISTILL	0.7772	0.0555	$+1.19 \pm 0.12$	0.7186	0.0760	$+0.16 \pm 0.05$
ULC [9]	0.7773	0.0529	$+1.18 \pm 0.22$	0.7188	0.0797	$+0.13 \pm 0.11$
FSIW [11]	0.7785	0.0553	$+1.03 \pm 0.26$	0.7193	0.0780	$+0.07 \pm 0.08$
DISTILL-ONLY	0.7821	0.0672	$+0.56 \pm 0.14$	0.7277	0.0744	-1.11 ± 0.10
FRESH	0.7929	0.1855	-0.80 ± 0.25	0.7357	0.1819	-2.20 ± 0.06
Single visited: no						
FTP [5]	0.7786	0.0515	$+1.02 \pm 0.18$	0.7162	0.0567	$+0.50 \pm 0.15$
PI [13]	0.7806	0.0692	$+0.76 \pm 0.43$	0.7177	0.0871	$+0.29 \pm 0.15$
DEFUSE [2]	0.7789	0.0681	$+0.98 \pm 0.30$	0.7201	0.0955	-0.04 ± 0.14
ES-DFM [10]	0.7791	0.0690	$+0.96 \pm 0.31$	0.7203	0.0966	-0.07 ± 0.13
DDFM [3]	0.7902	0.0632	-0.46 ± 0.31	0.7214	0.0562	-0.22 ± 0.20
DFM [1]	0.7569	0.0612	$+3.78 \pm 0.19$	0.7236	0.0578	-0.53 ± 0.04
MISS [6]	0.7830	0.0826	$+0.45 \pm 0.24$	0.7238	0.1292	-0.56 ± 0.20

all 12 cells survive Holm–Bonferroni correction, and 0.70% at $v = 1\,\mathrm{d}$. Figure 1 shows where that gain comes from and where it runs out. ORACLE, which writes at o but reads the matured label, gains 2.51% at $v = 7\,\mathrm{d}$, so little recoverable headroom is left above us, while FRESH, the same early write left uncorrected, loses 20.0%, an order of magnitude more than the gain above it. The correction, not the early write, carries the result. Two further features of the figure matter for deployment. The gain is largest where the funnel is slowest, because WAIT at $v = 7\,\mathrm{d}$ trains on week-old data and no amount of label accuracy compensates for that. And the two rows are near-identical: the result does not depend on which teacher supplies the estimate, a difference of $+0.022$ pp over all 24 cells, and freezing that teacher after initialisation so that it ships as a static file rather than a second training job costs 0.25 pp and still beats WAIT in 22 of 24 cells.

Which target to write depends on the served capacity. The two targets separate along the capacity axis. DISTILL-ONLY, which carries no observed label at all, beats WAIT in all 12 of 12 cells at the two smallest capacities, by 0.60% at $v = 1\,\mathrm{d}$ and 0.85% at $v = 7\,\mathrm{d}$, and loses at the two largest by 0.50% and 0.14%, crossing over between M_1 and M_2 at every join delay, plausibly because a soft

target acts as a regulariser that a larger model no longer needs [7]. Early-funnel stages such as retrieval can therefore drop the hard label entirely and emit a record of features and one number, whereas larger served models should keep the observed positive. Table 1 places both targets against the literature at one operating point, $v = 1\,\mathrm{d}$ and $o = 90$ minutes. Among the methods a single-visit pipeline can actually run we lead at both rungs, +1.19% against +1.18% for ULC [9] at M_0 and +0.16% against +0.13% at M_3, with overlapping intervals, and we are better calibrated than WAIT at both rungs while FRESH degrades calibration error by 3.2 and 2.4 times. The constraint is not free: DFM reaches +3.78% at M_0 by writing the same click several times, which supplies training signal that no single-visit target can recover.

4 Conclusion and Future Work

An objective's attribution window is fixed by the business definition of a conversion; the join delay its training pipeline adopts is not. We have shown that the two can be set independently without paying for it. Writing the record within minutes and replacing only its observed negatives with a teacher's estimate beats waiting for the matured label in 23 of 24 configurations and recovers 76% of what a label-peeking oracle attains at seven days, with the gain growing as the funnel slows, and it does so under the single-visit constraint that most published corrections violate. Which target to write is a function of the served capacity: a small model can drop the observed label entirely and train on the teacher's estimate alone, while a larger one should keep the observed positive. The correction is cheap to adopt, being insensitive to the teacher's architecture and still effective when the teacher is frozen after initialisation and shipped as a static file.

The next step is consolidating several ranking objectives into one model and one stream, an active industrial direction [8] whose first obstacle is a pipeline obstacle rather than a modelling one. Under WAIT, objective k's record exists only at $C_i + v_k$, so a shared append-only stream must either write K records per click, violating single-visit K times over, or delay every objective until $\max_k v_k$ and impose the slowest funnel's staleness on the fastest. The target studied here removes that dilemma in principle: a single stream at one short join delay can carry $\tilde{y}_{i,k} = Y_{i,k}^{(o)} + (1 - Y_{i,k}^{(o)})\, w_k^T(x_i)$ for every objective at once, because w_k^T is a function the pipeline evaluates rather than an outcome it awaits, and the teachers sit upstream as static files rather than K running training jobs. We ran one objective at a time, so the open question is interference: whether K heads sharing a trunk and K soft targets of differing sharpness still train stably. Settling it needs K prediction heads, K attribution windows, one stream, and a production-scale teacher, whose richer features should also sharpen the corrector beyond what a corpus of anonymised categorical fields allows.

References

1. Chapelle, O.: Modeling delayed feedback in display advertising. In: Proc. KDD. pp. 1097–1105 (2014)

2. Chen, Y., Jin, J., Zhao, H., Wang, P., Liu, G., Xu, J., Zheng, B.: Asymptotically unbiased estimation for delayed feedback modeling via label correction. In: Proc. WWW. pp. 369–379 (2022)
3. Dai, S., Zhou, Y., Xu, J., Wen, J.R.: Dually enhanced delayed feedback modeling for streaming conversion rate prediction. In: Proc. CIKM. pp. 390–399 (2023)
4. Ding, C., Wu, J., Yuan, Y., Li, C., Wang, X., Wang, D., Yang, F., Rabinovich, A.: Delayed feedback modeling with influence functions. Proc. AAAI **40**(17), 14648–14656 (2026)
5. Li, H., Pan, F., Ao, X., Yang, Z., Lu, M., Pan, J., Liu, D., Xiao, L., He, Q.: Follow the prophet: Accurate online conversion rate prediction in the face of delayed feedback. In: Proc. SIGIR. pp. 1915–1919 (2021)
6. Liu, Q., Ao, X., Guo, Y., He, Q.: Online conversion rate prediction via multi-interval screening and synthesizing under delayed feedback. Proc. AAAI **38**(8), 8796–8804 (2024)
7. Menon, et al.: A statistical perspective on distillation. In: Proc. ICML. PMLR, vol. 139 (2021)
8. Meta AI: Sequence learning: A paradigm shift for personalized ads recommendations. Meta AI Engineering Blog (2025), `https://ai.meta.com/blog/generative-ads-recommendation-model-gem/`
9. Wang, Y., Sun, P., Zhang, M., Jia, Q., Li, J., Ma, S.: Unbiased delayed feedback label correction for conversion rate prediction. In: Proc. KDD. pp. 2456–2466 (2023)
10. Yang, J.Q., Li, X., Han, S., Zhuang, T., Zhan, D.C., Zeng, X., Tong, B.: Capturing delayed feedback in conversion rate prediction via elapsed-time sampling. Proc. AAAI **35**(5), 4582–4589 (2021)
11. Yasui, S., Morishita, G., Fujita, K., Shibata, M.: A feedback shift correction in predicting conversion rates under delayed feedback. In: Proc. WWW. pp. 2740–2746 (2020)
12. Zhang, B., Luo, L., Chen, Y., Nie, J., Liu, X., Guo, D., Zhao, Y., Li, S., Hao, Y., Yao, Y., Lakshminarayanan, G., Wen, E.D., Park, J., Naumov, M., Chen, W.: Wukong: Towards a scaling law for large-scale recommendation (2024)
13. Zhang, X., Li, W., Li, R., Fu, Z., Tang, T., Zhang, Z., Chen, W.Y., Noorshams, N., Jasapara, N., Ding, X., Wen, E., Feng, X.: Personalized interpolation: Achieving efficient conversion estimation with flexible optimization windows. In: Proc. CIKM (2025)